



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS

ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

GCC PLUGIN FOR STATIC ANALYZER SUPPORT

GCC PLUGIN PRO STATICKÉ ANALYZÁTORY

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

LUKÁŠ PŠEJA

SUPERVISOR

VEDOUCÍ PRÁCE

Dr. Ing. PETR PERINGER,

BRNO 2026

Bachelor's Thesis Assignment



172017

Institut: Department of Intelligent Systems (DITS)
Student: **Pšeja Lukáš**
Programme: Information Technology
Title: **GCC plugin for static analyzer support**
Category: Software analysis and testing
Academic year: 2025/26

Assignment:

1. Get acquainted with *CodeListener* component from the *Predator* project. Analyze the implementation of *CodeListener* and similar projects. Additionally, study the relevant data structure visualization concepts.
2. Design a new GCC plugin as a modern alternative to *CodeListener* and add new functionality (built-in visualization support and export in JSON format). Focus on improvement of the plugin interface.
3. Implement the plugin in C++ along with appropriate examples for demonstration purposes. Verify the plugin's compatibility by using it with *Predator*.
4. Test the plugin using the created examples with various GCC versions. Evaluate obtained results.

Literature:

- GitHub repository of Predator project.
<https://github.com/kdudka/predator>

Requirements for the semestral defence:

- First two points of assignment.

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Peringer Petr, Dr. Ing.**
Head of Department: Kočí Radek, Ing., Ph.D.
Beginning of work: 1.11.2025
Submission deadline: 13.5.2026
Approval date: 3.11.2025

Abstract

This bachelor's thesis deals with the analysis of the Code Listener infrastructure, which serves as the foundation for creating static analyzers, and its subsequent refactoring into a new plugin.

Abstrakt

Tato bakalářská práce se zabývá analýzou infrastruktury Code Listener, která poskytuje základ pro tvorbu statických analyzátorů zdrojového kódu, a její následnou refaktorizací do nového pluginu.

Keywords

Code Listener infrastructure, GCC, LLVM, plugin, refactoring, static analysis, C, C++

Klíčová slova

infrastruktura Code Listener, GCC, LLVM, plugin, refaktorování, statická analýza, C, C++

Reference

PŠEJA, Lukáš. *GCC plugin for static analyzer support*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Dr. Ing. Petr Peringer,

GCC plugin for static analyzer support

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Dr. Ing. Petr Peringer. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used Grammarly to correct spelling and other language mistakes.

.....

Lukáš Pšeja
April 24, 2026

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Thesis structure	4
2	State of the Art	5
2.1	The Code Listener Infrastructure	5
2.2	The GCC Architecture and GIMPLE Representation	7
2.3	Code Listener Internal Architecture	8
2.4	Limitations of the Legacy Architecture	10
3	Design	12
4	Implementation	13
5	Testing and Evaluation	14
6	Conclusion	15
	Bibliography	16
A	Build and Usage Instructions	17

List of Figures

2.1	Block diagram of the legacy Code Listener infrastructure. The system is divided into interconnected modules, beginning with the code parser interface, which extracts intermediate representations from compilers such as GCC or Sparse. This data flows through optional filters, such as <code>switch to if</code> transformations, and listeners like the CFG plotters before being aggregated into a persistent object model within the code storage component. Finally, static analyzers such as Predator and Forester can query this code storage to perform formal verification, routing any discovered defects back through the code parser interface and the code parser's error stream to maintain consistent compiler-like error reporting[1].	6
2.2	Control Flow Graph (CFG) representation of functions within the Code Listener Intermediate Representation. The graph models the execution structure using basic blocks as vertices and control flow transitions as directed edges. Each basic block contains a strictly sequential list of non-terminal instructions, such as function calls (<code>CALL</code>) or binary operations (<code>BINOP</code>). To determine the subsequent flow of execution, every block concludes with exactly one terminal instruction, such as an unconditional jump (<code>JMP</code>), conditional jump (<code>COND</code>), or return (<code>RET</code>) [1].	9

Chapter 1

Introduction

This bachelor’s thesis focuses on the design and implementation of a modernized reimplementation of the Code Listener infrastructure, a C++ library that serves as a vital bridge between compilers and static analysis tools, particularly the Predator shape analysis tool suite. The original Code Listener architecture, while successful in enabling static analysis of C programs that manipulate heap memory, exhibits several design limitations that have accumulated over time. This thesis aims to address these constraints by replacing the callback-driven pipeline with a persistent Code Model, demoting analyses from hard-wired pre-analysis steps to optional, on-demand annotation services, and introducing a structured query interface and uniform support for exporting results in common formats.

1.1 Motivation

Software development is a complex and error-prone process, especially as software systems grow in size and complexity. As software systems evolve, certain categories of defects become undetectable solely through testing, as test suites can only evaluate explicitly covered execution paths, leaving much of the program’s state space unexplored.

Nowhere is this limitation more critical than in low-level system programming languages such as C and C++, where developers bear the burden of manual memory management. In these environments, insidious bugs such as memory leaks, buffer overflows, use-after-free errors, and dangling pointer dereferences frequently hide within unexplored execution paths. The consequences of these hidden defects can be catastrophic, particularly in mission-critical domains like aerospace engineering. While rigorous testing prevents many fatal accidents, memory mismanagement has repeatedly compromised modern aircraft. For instance, regulatory agencies have had to issue mandatory airworthiness directives for both the Boeing 787 and the Airbus A350, requiring airlines to periodically reboot the aircraft to clear memory buffers and prevent the total mid-flight loss of critical systems. These near-misses underscore a fundamental reality: traditional dynamic testing is insufficient to guarantee memory safety in high-stakes environments.

To prevent such failures, developers must rely on advanced static analysis tools, such as the Predator shape analysis suite, which is capable of formally verifying a program’s state space without executing it. However, developing and maintaining these analyzers presents a significant architectural challenge: extracting the program’s semantics from the compiler’s internal representation. Analyzers must interface with compilers to understand the code, but native compiler APIs are notoriously complex, tightly coupled to specific versions, and

largely undocumented. Building analysis tools directly on top of these raw APIs leads to brittle, maintenance-heavy software.

To overcome this bottleneck, a clean abstraction layer is required. Specifically, researchers need a modular middleware that can abstract away the massive, often undocumented codebases of industrial compilers, providing instead a concise, unified, and well-documented object-oriented API. This ensures that static analysis tools can be developed independently of the underlying compiler architecture.

1.2 Thesis structure

The remainder of this thesis is structured as follows. Chapter 2 provides a deeper technical analysis of the legacy Code Listener architecture. Chapter 3 outlines the architectural modernization, detailing the transition from a callback-driven pipeline to a persistent Code Model. Chapter 4 documents the implementation process, including development environment setup, portability considerations, and code metrics regarding the refactored infrastructure. Chapter 5 presents the methodology and results of regression testing, demonstrating the correctness and performance of the new implementation against the existing test suite. Finally, Chapter 6 summarizes the contributions of this work and outlines potential avenues for future research and development.

Chapter 2

State of the Art

This chapter provides a detailed account of the existing Code Listener infrastructure in the Predator repository [7, 6] and as described in the original publication [1]. Understanding the design decisions and limitations of the current implementation is necessary background for the design choices made in this thesis.

2.1 The Code Listener Infrastructure

The Code Listener (CL) infrastructure is an open-source middleware designed to simplify the construction of static analysis tools for C [1] and, by extension, C++ [3] programs. It was developed by the VeriFIT research group¹ at Brno University of Technology mainly in the years 2009–2015, with some contributors continuing work on it until the present². Its primary goal is to wrap the complex and often undocumented interfaces of existing code parsers to provide a unified, well-documented, and object-oriented Application Programming Interface (API) for static analysis tool developers [1]. By transforming the native internal representations of compilers into its own simplified format, the Code Listener API exposed to static analyzers becomes completely independent of the underlying parser. This abstraction layer allows the same analysis tool to run across different compilers without requiring modifications to the analyzer’s source code [1].

The architecture of the original Code Listener is highly modular, effectively acting as a bridge between the compiler’s internal representations and the static analysis tools, as illustrated in Figure 2.1 on page 6, and is composed of several key components:

- **Code Parser Interface** is a layer that communicates directly with the source code parsers, such as the GCC compiler. Small adapters embedded within the code parsers are responsible for translating the parser’s specific intermediate representation into the parser-independent representation required by Code Listener.
- **Filters and Listeners** are components that process intermediate code once it is obtained through the parser interface. Filters perform various transformations on the intermediate code, for example, breaking down a complex `SWITCH` instruction into a series of simpler `COND` instructions. Listeners, on the other hand, observe the stream of events generated by the code parser interface and can be used to provide diagnostic

¹<https://www.fit.vut.cz/research/group/verifit/.en>

²<https://github.com/kdudka/predator/graphs/contributors>

information or to generate visualizations, such as control flow graph (CFG) plotters, or intermediate code printers.

- **Code Storage** aggregates the incoming data to build a persistent, serialized object model of the analyzed source code, because advanced static analyzers often need to traverse the program structure repeatedly, which would be inefficient if they had to rely on a simple callback-driven pipeline. This component allows the analyzers to look up containers of types, variables, and functions, and to navigate the CFG of each function.
- **Analyzers** are the static analysis tools that can query the API and begin their formal verification operations once the code storage builds the complete object model. Any diagnostic messages emitted by the analyzers are passed back to the code parser interface, which formats them in the standard way expected by the compiler, including file names and line numbers, so that they can be easily understood by developers.

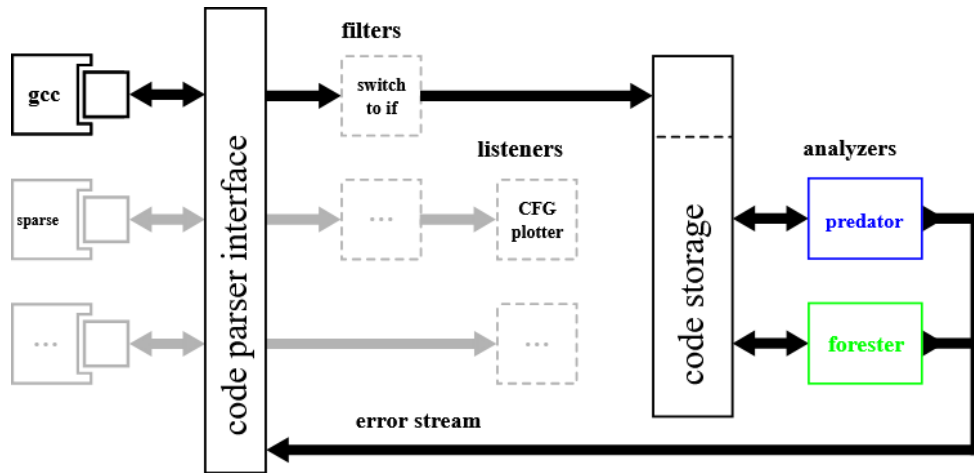


Figure 2.1: Block diagram of the legacy Code Listener infrastructure. The system is divided into interconnected modules, beginning with the code parser interface, which extracts intermediate representations from compilers such as GCC or Sparse. This data flows through optional filters, such as `switch to if` transformations, and listeners like the CFG plotters before being aggregated into a persistent object model within the code storage component. Finally, static analyzers such as Predator and Forester can query this code storage to perform formal verification, routing any discovered defects back through the code parser interface and the code parser’s error stream to maintain consistent compiler-like error reporting[1].

The utility of the Code Listener infrastructure is demonstrated by the advanced verification tools that rely on it. The two primary analyzers built on top of Code Listener are Predator [7] and Forester [1]. Predator is a tool for automated formal verification of low-level C programs that manipulate dynamic, pointer-like data structures, such as linked lists[9]. Forester focuses on verifying programs that manipulate complex dynamic data structures using tree automata[8]. By using Code Listener as a compiler plugin, these tools can seamlessly analyze anything the compiler can process. This ensures that the user does not need to manually preprocess source code or modify standard build systems to run the

analyzers [1], which is a significant advantage for usability and integration with existing development workflows.

Similar Projects and Ecosystem Comparisons

To contextualize the architectural choices of the Code Listener infrastructure, it is valuable to compare it against peer projects in the static analysis domain.

The primary peer to Code Listener is the Frama-C platform, an extensible, open-source framework dedicated to the static analysis of C source code [4]. Much like Code Listener, Frama-C isolates its analysis plug-ins from the raw compiler to provide a unified Application Programming Interface. It achieves this by using a modified version of the CIL (C Intermediate Language) front-end, which parses ISO C99 programs into a normalized Abstract Syntax Tree (AST) [4]. Frama-C utilizes a highly collaborative architecture where various plug-ins, such as those based on abstract interpretation or deductive verification, share this AST and communicate via ACSL (ANSI/ISO C Specification Language) annotations [4]. While Frama-C operates primarily at the AST level, Code Listener translates compiler internals into a reduced, instruction-based control flow graph. The key advantage of Code Listener’s approach is that it allows building of analyzers capable of handling everything industrial compilers are able to compile [1], whereas Frama-C is limited to the subset of C that can be represented in its AST and supported by the CIL front-end [4].

Other notable comparisons can be drawn from the LLVM compiler infrastructure. The Clang Static Analyzer, built on top of the Clang/LLVM architecture, provides an extensible framework for checking C and C++ code, though historically its scope has been more limited to compilation and AST-level linting than Frama-C’s exhaustive verification [4]. However, the LLVM core provides a highly optimized, typed, Static Single Assignment (SSA) intermediate representation [5]. Notably, modern LLVM utilizes an **AnalysisManager** approach that elegantly solves the “God Object” anti-pattern observed in the legacy Code Listener, which is described in Section 2.4 on page 11. Instead of polluting the core Intermediate Representation (IR) with analysis results, LLVM’s **AnalysisManager** computes results lazily and stores them externally, keyed by the IR node identity [5]. This clean separation of concerns heavily inspires the modernized architecture proposed in this thesis.

2.2 The GCC Architecture and GIMPLE Representation

To understand how the Code Listener extracts program semantics, it is essential to examine the internal architecture of the GNU Compiler Collection (GCC). GCC operates as a sequential pipeline divided into three primary blocks: the front end, the middle end, and the back end, and uses three main intermediate languages to represent a program during compilation, specifically GENERIC, GIMPLE, and RTL [2].

The front end is responsible for the lexical analysis and parsing of a specific programming language. It generates GENERIC, a common, language-independent representation that can represent programs written in all languages supported by GCC [2]. GENERIC serves as the standardized interface between the parser and the optimizer [2].

The middle end optimizes the program using the GIMPLE representation [2]. The compiler pass responsible for converting GENERIC into GIMPLE is called the *gimplifier* [2]. At this stage, GCC also supports loadable modules called plugins, which are invoked at pre-determined locations in the compilation process to provide extra features to the compiler [2]. The Code Listener infrastructure utilizes this plugin API, registering callbacks

for specific events to intercept the compilation process and access the intermediate code without modifying the compiler itself [1].

Finally, the back end uses RTL (Register Transfer Language) to perform architecture-dependent optimizations and generate the assembly code for the specific target hardware [2].

The GIMPLE Intermediate Language

When intercepting the compilation pipeline, the Code Listener adapter parses the GIMPLE representation [1]. GIMPLE is a tree-based representation derived from GENERIC, specifically designed to facilitate target- and language-independent optimizations [2].

A defining characteristic of GIMPLE is its highly restrictive grammar. Expressions are broken down into a three-address representation consisting of tuples containing no more than three operands, with the exception of function calls [2]. To accommodate the breakdown of complex expressions, the compiler introduces artificial temporary variables to hold intermediate values [2]. Furthermore, expressions with side effects are strictly limited to the right-hand side of assignments [2].

Additionally, GIMPLE drastically simplifies the program’s control flow. All high-level control structures used in GENERIC are dismantled and lowered into conditional jumps [2]. Lexical scopes are removed entirely, and exception regions are extracted and converted into an on-the-side exception region tree [2].

By flattening source code into this concise, explicit-control-flow format, GIMPLE provides a highly predictable foundation. The Code Listener adapter systematically translates these GIMPLE trees into its own Code Listener IR (CL IR), utilizing a custom control flow graph to fully decouple the static analyzers from the compiler’s internal complexities [1].

2.3 Code Listener Internal Architecture

To decouple static analysis tools from the GCC compiler, the Code Listener adapter systematically translates GIMPLE into its own simplified format, the Code Listener Intermediate Representation (CL IR). In this representation, the execution structure of each parsed function is modeled using a Control Flow Graph (CFG) [1].

A Control Flow Graph is a directed mathematical graph where the vertices represent basic blocks, and the edges represent the possible transfers of control flow during program execution [1]. A basic block is defined as a straight-line sequence of code with exactly one entry point and one exit point. Once execution enters a basic block, all instructions within it are executed sequentially, without branching or halting, until the end of the block is reached.

To construct these blocks, the Code Listener API restricts its intermediate instruction set to two distinct groups:

- **Non-terminal instructions** which form the body of a basic block and cannot be used to exit it [1]. The set is highly reduced, consisting only of unary operations (UNOP), binary operations (BINOP), and function calls (CALL) [1].
- **Terminal instructions** are strictly located at the very end of a basic block and are responsible for determining the outgoing edges of the CFG [1]. This group includes unconditional jumps (JMP), conditional jumps (COND), switch statements (SWITCH), function returns (RET), and program aborts (ABORT) [1].

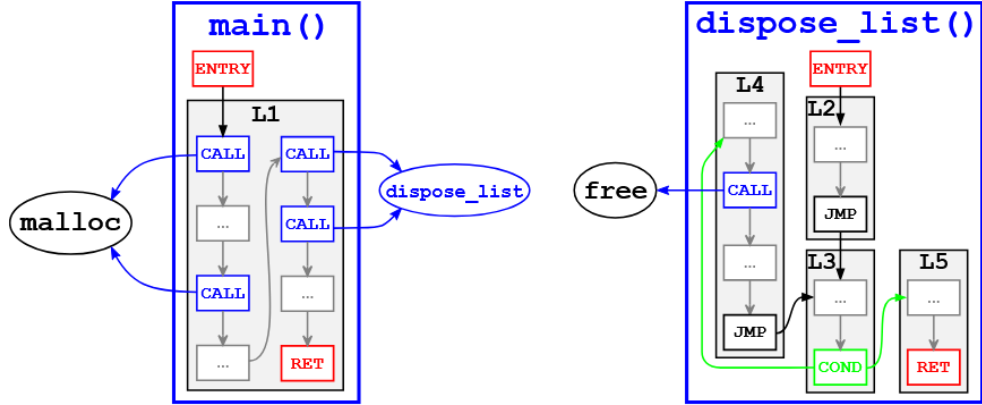


Figure 2.2: Control Flow Graph (CFG) representation of functions within the Code Listener Intermediate Representation. The graph models the execution structure using basic blocks as vertices and control flow transitions as directed edges. Each basic block contains a strictly sequential list of non-terminal instructions, such as function calls (CALL) or binary operations (BINOP). To determine the subsequent flow of execution, every block concludes with exactly one terminal instruction, such as an unconditional jump (JMP), conditional jump (COND), or return (RET) [1].

Instructions manipulate data through operands, represented by the `cl_operand` structure. An operand must refer either to a variable, such as a local or global variable, or a constant, such as a numeric literal, a string, or a function definition [1]. To alter the semantic meaning of an operand, for instance, to dereference a pointer, access an array element, or read a specific field within a composite structure, Code Listener uses accessors (`cl_accessor`) [1]. To guarantee simplicity for the analysis tools, Code Listener enforces a strict rule: the chaining of multiple dereferences is not allowed, and whenever a multiple dereference appears in the original C source code, the adapter automatically breaks it down into a sequence of separate instructions, ensuring each operand contains at most one dereference [1].

Furthermore, the C programming language is statically typed, and Code Listener preserves all type information known at compile-time. Every operand and accessor is explicitly associated with a type (`cl_type`), and composite types are defined recursively based on their constituent elements [1]. Because variable names in C source code are often ambiguous due to lexical scoping, Code Listener assigns a globally unique integer identifier (`uid`) to every variable and function [1].

Ultimately, the incoming stream of instructions, operands, and types is aggregated to build a persistent, serialized object model known as the Code Storage [1]. This C++ API provides the static analyzers with look-up databases for types (`TypeDb`), variables (`VarDb`), and functions (`FncDb`), effectively delivering a complete, parser-independent representation of the program ready for formal verification.

Data Structure Visualization Concepts

Static analysis tools extract and generate massive amounts of complex semantic data. To ensure that the Code Listener correctly translates the compiler’s internal representation and to allow users to interpret analysis results, robust data-structure visualization is required.

Program Graph Visualization

The execution structure of a program is inherently non-linear and is best modeled mathematically as a directed graph. Consequently, visualizing the program’s Control Flow Graph (CFG) and Call Graph is crucial for inspecting what the infrastructure has extracted. By outputting these structures into standard graph description languages, such as the DOT format, developers can generate visual plots of the basic blocks, terminal instructions, and branching paths [1]. This provides immediate visual confirmation that the adapter has successfully mapped the compiler’s intermediate code without losing structural integrity.

Heap Shape Visualization

Beyond control flow, visualization is particularly vital for the specific domain of the Predator shape analyzer. Predator verifies programs that manipulate dynamically linked data structures by using an abstract interpretation domain based on Symbolic Memory Graphs (SMGs) [7]. Because tracking infinite sets of heap configurations, such as cyclic doubly-linked lists, is highly complex, Predator can produce error traces in a graphical format [7]. Visualizing these heap shapes enables analysts to graphically track memory leaks, invalid pointer dereferences, and the layout of the memory, bridging the gap between abstract mathematical models and human comprehension.

2.4 Limitations of the Legacy Architecture

While the legacy Code Listener infrastructure successfully abstracts the compiler’s internal representation and has enabled tools like Predator to achieve high precision and win several awards [9], its architectural design exhibits several critical limitations. These constraints primarily stem from its inflexible filter pipeline, eager analysis execution, and tight coupling of analysis results to the core data structures.

Hardwired Filter Pipeline

The legacy GCC plugin configures its listener chains, such as diagnostic printers, graph plotters, and the analyzer itself, before the compilation process begins. Consequently, a predetermined set of intermediate filters, such as the `unfold_switch` filter, which converts `SWITCH` instructions into chains of conditional jumps, is hardwired for all listener invocations. This presents a binary choice: static analyzers have no runtime mechanism to opt out of these transformations, preventing tools from analyzing the raw, unfiltered control flow graph when needed.

Unconditional Pre-Analyses

The standard entry point for an analyzer within the legacy framework is the `CLEasy` interface. Before this interface hands control over to the implemented analyzer, it unconditionally executes four mandatory pre-analyses: building the call graph, finding loop-closing edges, performing a points-to analysis, and resolving variable liveness. Because there is no lazy evaluation or feature-flag mechanism, every analyzer is forced to pay the full computational cost of these operations, even for a trivial tool, such as a smoke test, that requires none of the resulting data.

IR Pollution and the God Object Anti-Pattern

The root cause of the mandatory pre-analyses is the design of the Code Storage data structures. The legacy architecture violates the Single Responsibility Principle by embedding the results of pre-analyses directly as member fields within the core Intermediate Representation (IR) nodes, including the **Insn**, **Fnc**, and **Storage** structures. For instance, a function node (**Fnc**) acts simultaneously as the IR representation of a function, a points-to analysis result container, and a call-graph membership record. This tightly couples the program representation with analysis data, creating circular ownership: a function owns a points-to graph that holds a raw pointer back to the function. Because these analysis results have no external storage mechanism, they must be eagerly computed and pre-populated into the IR before handoff.

One-Way Handoff

Which brings us to the final architectural limitation: the legacy pipeline culminates in a strict, one-way handoff. The analyzer is provided with a read-only reference to the fully populated code model. Once execution enters the analyzer, it has no structured query interface to communicate results back to the framework, nor can it dynamically request auxiliary services. For example, if an analyzer discovers a suspicious function during execution and wishes to generate a visual DOT dump of its control flow graph, it cannot do so because diagnostic outputs must be strictly pre-configured via compiler flags at invocation time.

To resolve these constraints, a modernized architecture is required. One that decouples the core IR from analysis data, evaluates annotation services on demand, and provides a flexible query interface. The design of this modernized infrastructure is the subject of the following chapter.

Chapter 3

Design

Chapter 4

Implementation

Chapter 5

Testing and Evaluation

Chapter 6

Conclusion

Bibliography

- [1] DUDKA, K.; PERINGER, P. and VOJNAR, T. An Easy to Use Infrastructure for Building Static Analysis Tools. *Lecture Notes in Computer Science*, 2012, vol. 2012, no. 6927, p. 527–534. ISSN 0302-9743. Available at: <http://www.springerlink.com/content/75024011tk386572/>.
- [2] *GCC Internals* online. Available at: <https://gcc.gnu.org/onlinedocs/gccint/>. [cit. 2026-04-04].
- [3] KAŠPAR, D. *Extension of the Code Listener Infrastructure Adding C++ Support*. Brno, Czech Republic, 2015. Bachelor’s Thesis. Brno University of Technology, Faculty of Information Technology.
- [4] KIRCHNER, F.; KOSMATOV, N.; PREVOSTO, V.; SIGNOLES, J. and YAKOBOWSKI, B. Frama-C: A software analysis perspective. *Formal Aspects of Computing*, May 2015, vol. 27, no. 3, p. 573–609. ISSN 1433-299X. Available at: <https://doi.org/10.1007/s00165-014-0326-7>.
- [5] LATTNER, C. and ADVE, V. LLVM: a compilation framework for lifelong program analysis & transformation. In: *International Symposium on Code Generation and Optimization, 2004. CGO 2004*. 2004, p. 75–86.
- [6] PERINGER, P.; ŠOKOVÁ, V. and VOJNAR, T. PredatorHP Revamped (Not Only) for Interval-Sized Memory Regions and Memory Reallocation (Competition Contribution). In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2020), Part II*. Springer, 2020, vol. 12079, p. article, Competition Contribution. Lecture Notes in Computer Science.
- [7] PERINGER, P.; ŠOKOVÁ, V.; TRTÍK, M.; VOJNAR, T.; HOLÍK, L. et al. Predator Shape Analysis Tool Suite. In: *Proceedings of HVC 2016*. Zurich: Springer International Publishing, 2016, vol. 10028, p. 202–209. Lecture Notes in Computer Science. ISBN 978-3-319-49052-6. Available at: http://link.springer.com/chapter/10.1007/978-3-319-49052-6_13.
- [8] VERIFIT RESEARCH GROUP. *Forester* <https://www.fit.vut.cz/research/group/verifit/public/tools/forester/>. [Cit. 2026-04-23].
- [9] VERIFIT RESEARCH GROUP. *Predator* <https://www.fit.vut.cz/research/group/verifit/public/tools/predator/>. [Cit. 2026-04-23].

Appendix A

Build and Usage Instructions

Requirements

- GCC 12 (gcc-12, g++-12, gcc-12-plugin-dev)
- CMake 3.28+
- patch (used to apply the Predator shim patch)
- Internet access at configure time (nlohmann/json fetched via `FetchContent`)

On Debian/Ubuntu:

```
sudo apt install gcc-12 g++-12 gcc-12-plugin-dev cmake patch
```

Clone

```
git clone --recurse-submodules \
  git@github.com:pseja/gcc-plugin-for-static-analyzer-support.git
cd gcc-plugin-for-static-analyzer-support
```

Configure and Build

```
make configure # cmake -B build -DTARGET_GCC=gcc-12
make build     # cmake --build build -j$(nproc)
```

Key build outputs in build/:

- `libcl_gcc.so` — the GCC plugin
- `libcl_json_dump.so` — JSON dump analyzer
- `libcl_callgraph_dot.so` — call-graph DOT analyzer
- `libcl_recursion_check.so` — recursion detector
- `cl_merge`, `cl_analyze` — standalone tools
- `analyzers/predator/sl_build/libsl_analyzer.so` — Predator (if `WITH_PREDATOR=ON`)

Run Tests

```
make test
```

Run an Analyzer (Single TU)

```
gcc-12 -S file.c -o /dev/null \  
-fplugin=./build/libcl_gcc.so \  
-fplugin-arg-libcl_gcc-load-analyzer=./build/libcl_json_dump.so \  
-fplugin-arg-libcl_gcc-args=file.json
```

Whole-Program Analysis

```
# 1. compile each translation unit to a JSON model  
gcc-12 -S a.c -o /dev/null -fplugin=./build/libcl_gcc.so \  
-fplugin-arg-libcl_gcc-load-analyzer=./build/libcl_json_dump.so \  
-fplugin-arg-libcl_gcc-args=a.json  
# ... repeat for b.c -> b.json, c.c -> c.json  
  
# 2. merge  
./build/cl_merge a.json b.json c.json -o merged.json  
  
# 3. analyze  
./build/cl_analyze merged.json \  
--analyzer=./build/libcl_callgraph_dot.so \  
--args=callgraph.dot
```